Ειδικά Θέματα Αλγορίθμων: Μηχανική Μάθηση
Δεύτερο Πακέτο Ασκήσεων

Παναγή Αντρέας

Τμήμα Μαθηματικών
Εθνικόν και Καποδιστριακόν Πανεπιστήμιον Αθηνών
Αθήνα, Ελλάδα *

Ιούλιος 2024

ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
Εθνικόν και Καποδιστριακόν
Πανεπιστήμιο Αθηνών

∗Στοιχειοδέτηση με χρήση XΕLATEX

Περιεχόμενα

Άσκηση 1

Εκφώνηση

Στην άσκηση αυτή ζητείται να αναλύσετε τα δεδομένα στο αρχείο dropout_prediction, που αφορούν την πρόβλεψη επιτυχούς ή όχι ολοκλήρωσης των σπουδών από μια μελέτη για μαθητές μέσης εκπαίδευσης στην Πορτογαλία.

Η λεπτομερής περιγραφή των μεταβλητών βρίσκεται στην ιστοσελίδα
https://archive.ics.uci.edu/dataset/697/predict+students+dropout+and+academic+success
ενώ η περιγραφή της μελέτης στο άρθρο paper1.pdf που επισυνάπτεται.

Ζητείται να εφαρμόσετε διάφορα μοντέλα πρόβλεψης για τη μεταβλητή target η οποία περιγράφει το αποτέλεσμα των σπουδών του μαθητή (success=αποφοίτηση στην κανονική διάρκεια, relative_success=αποφοίτηση με έως 3 χρόνια καθυστέρηση και failure=εγκατάλειψη ή μη αποφοίτηση μέσα στα 3 χρόνια).

Μπορείτε να δοκιμάσετε : 1. Logistic regression και lasso, θέτοντας παίρνοντας δύο κατηγορίες : success και relative success ή failure. 2. Linear and Quadratic Discriminant analysis 3. Random Forests 4. Neural Networks

Σε κάθε κατηγορία μπορείτε να πειραματιστείτε με τιμές των υπερπαραμέτρων, δομή του δικτύου κλπ. Για χωρισμό σε training/test set, πάρτε ένα τυχαίο δείγμα μεγέθους 20% του συνόλου για testing. Αν κάνετε cross validation εφαρμόστε την στο training set.

Τα αποτελέσματα της μελέτης σας αναφέρονται στη σύγκριση των διαφόρων μεθόδων που εφαρμόσατε ως προς το σφάλμα πρόβλεψης. Αναφέρετε τα συμπεράσματά σας σε μια σύντομη αναφορά, και περιλάβετε τους κώδικες που αναπτύξατε σε παράρτημα (ή δεύτερο κεφάλαιο αν χρησιμοποιήσετε R-markdown).

Σημείωση : Δεν είναι απαραίτητο τα συμπεράσματά σας να ταυτίζονται απόλυτα με αυτά στο άρθρο, επειδή, όπως θα δείτε, οι συγγραφείς αναφέρουν ότι έχουν χρησιμοποιήσει και μεθόδους data augmentation για να πετύχουν καλύτερη ισορροπία μεταξύ των κατηγοριών. Δεν ζητείται να κάνετε και εσείς κάτι τέτοιο. Όμως στην εργασία που θα παραδώσετε πρέπει να δίνονται αναφορές σε όποιες εργασίες, ιστοσελίδες, κλπ, χρησιμοποιήσατε.

Λύση

Βιβλιοθήκες, Βάση Δεδομένων και Επιθεώρηση Βάσης Δεδομένων

[6]:
```
#1 libraries
import numpy as np
import pandas as pd
import math
from IPython.display import display
from matplotlib.pyplot import subplots

import statsmodels.api as sm

from ISLP import load_data
from ISLP.models import (ModelSpec as MS ,
```

```
                              summarize)
from ISLP import confusion_table
from ISLP.models import contrast

from sklearn. discriminant_analysis import \
    (LinearDiscriminantAnalysis as LDA ,
     QuadraticDiscriminantAnalysis as QDA)
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression, ElasticNetCV


from sklearn.model_selection import train_test_split, ShuffleSplit, KFold,␣
↪cross_validate

from sklearn.preprocessing import OneHotEncoder
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
↪f1_score, confusion_matrix

from sklearn.tree import ( DecisionTreeClassifier as DTC ,
                           DecisionTreeRegressor as DTR ,
                           plot_tree ,
                           export_text)
from sklearn.metrics import (accuracy_score ,
                             log_loss)
from sklearn.ensemble import (RandomForestRegressor as RFR,
                              GradientBoostingRegressor as GBR,
                              RandomForestClassifier as RFC,
                              GradientBoostingClassifier as GBC)
```

[7]:
```
#2 Database Library
from ucimlrepo import fetch_ucirepo

# Fetch dataset
predict_students_dropout_and_academic_success = fetch_ucirepo(id=697)

# Data
X = predict_students_dropout_and_academic_success.data.features
y = predict_students_dropout_and_academic_success.data.targets

# Remove Enrolled - useless for prediction
mask = y['Target'] != 'Enrolled'
X= X[mask]
y= y[mask]
```

[8]:
```
#3 Inspect the data
# Get column names
print(X.dtypes)
print(X.columns)
print(y.columns)

# View random sample of rows (e.g. 10) to check data variability
```

```
print(X.sample(5))
print(y.sample(5))

# If you want to see unique values in each column (useful for categorical variables)
# for col in X.columns:
#     print(f"{col}: {X[col].unique()}\n")
```

```
Marital Status                                    int64
Application mode                                  int64
Application order                                 int64
Course                                            int64
Daytime/evening attendance                        int64
Previous qualification                            int64
Previous qualification (grade)                  float64
Nacionality                                       int64
Mother's qualification                            int64
Father's qualification                            int64
Mother's occupation                               int64
Father's occupation                               int64
Admission grade                                 float64
Displaced                                         int64
Educational special needs                         int64
Debtor                                            int64
Tuition fees up to date                           int64
Gender                                            int64
Scholarship holder                                int64
Age at enrollment                                 int64
International                                     int64
Curricular units 1st sem (credited)               int64
Curricular units 1st sem (enrolled)               int64
Curricular units 1st sem (evaluations)            int64
Curricular units 1st sem (approved)               int64
Curricular units 1st sem (grade)                float64
Curricular units 1st sem (without evaluations)    int64
Curricular units 2nd sem (credited)               int64
Curricular units 2nd sem (enrolled)               int64
Curricular units 2nd sem (evaluations)            int64
Curricular units 2nd sem (approved)               int64
Curricular units 2nd sem (grade)                float64
Curricular units 2nd sem (without evaluations)    int64
Unemployment rate                               float64
Inflation rate                                  float64
GDP                                             float64
dtype: object
Index(['Marital Status', 'Application mode', 'Application order', 'Course',
       'Daytime/evening attendance', 'Previous qualification',
       'Previous qualification (grade)', 'Nacionality',
       'Mother's qualification', 'Father's qualification',
       'Mother's occupation', 'Father's occupation', 'Admission grade',
       'Displaced', 'Educational special needs', 'Debtor',
       'Tuition fees up to date', 'Gender', 'Scholarship holder',
```

```
       'Age at enrollment', 'International',
       'Curricular units 1st sem (credited)',
       'Curricular units 1st sem (enrolled)',
       'Curricular units 1st sem (evaluations)',
       'Curricular units 1st sem (approved)',
       'Curricular units 1st sem (grade)',
       'Curricular units 1st sem (without evaluations)',
       'Curricular units 2nd sem (credited)',
       'Curricular units 2nd sem (enrolled)',
       'Curricular units 2nd sem (evaluations)',
       'Curricular units 2nd sem (approved)',
       'Curricular units 2nd sem (grade)',
       'Curricular units 2nd sem (without evaluations)', 'Unemployment rate',
       'Inflation rate', 'GDP'],
      dtype='object')
Index(['Target'], dtype='object')
      Marital Status  Application mode  Application order  Course  \
3599               1                43                  1    9147
2540               1                 7                  1    9147
2233               1                43                  1    9254
4296               1                44                  1    9070
4130               1                 1                  5    9500

      Daytime/evening attendance  Previous qualification  \
3599                           1                       1
2540                           1                       3
2233                           1                       6
4296                           1                      39
4130                           1                       1

      Previous qualification (grade)  Nacionality  Mother's qualification  \
3599                           133.1            1                       4
2540                           120.0            1                       1
2233                           138.6            1                      34
4296                           120.0            1                       1
4130                           143.0            1                      37

      Father's qualification  …  \
3599                      19  …
2540                      39  …
2233                      34  …
4296                      38  …
4130                      37  …

      Curricular units 1st sem (without evaluations)  \
3599                                               1
2540                                               0
2233                                               0
4296                                               0
4130                                               0

      Curricular units 2nd sem (credited)  \
```

```
3599                                    2
2540                                    0
2233                                    0
4296                                    5
4130                                    0

      Curricular units 2nd sem (enrolled)  \
3599                                    6
2540                                    5
2233                                    6
4296                                    9
4130                                    8

      Curricular units 2nd sem (evaluations)  \
3599                                      7
2540                                      5
2233                                     11
4296                                     10
4130                                      8

      Curricular units 2nd sem (approved)  Curricular units 2nd sem (grade)  \
3599                                    6                         14.500000
2540                                    0                          0.000000
2233                                    3                         11.666667
4296                                    7                         12.142857
4130                                    8                         14.448750

      Curricular units 2nd sem (without evaluations)  Unemployment rate  \
3599                                               0               9.4
2540                                               0              12.4
2233                                               0               7.6
4296                                               0              12.4
4130                                               0              15.5

      Inflation rate   GDP
3599            -0.8 -3.12
2540             0.5  1.79
2233             2.6  0.32
4296             0.5  1.79
4130             2.8 -4.06

[5 rows x 36 columns]
        Target
3726   Dropout
3634  Graduate
3254  Graduate
4348  Graduate
3324   Dropout
```

Επεξεργασία Δεδομένων

Καθορισμός ελεύθερων και εξαρτημένων μετβλητών καθώς και optional κωδικοποίηση

[9]:
```
#4 Prepare the data for ML models
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2,
                                                    random_state=42,
                                                    shuffle=True,
                                                    stratify=y) #needed in␣
↪classification to maintain class distribution

allvars=X_train.columns
design=MS(allvars)

X_train=design.fit_transform(X_train)
X_test = design.transform(X_test)
```

[10]:
```
# OPTIONAL Encode the categorial variables

X_train2, X_test2, y_train2, y_test2 = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

cat_cols = [
    'Marital Status',
    'Application mode',
    'Course',
    'Daytime/evening attendance',
    'Previous qualification',
    'Nacionality',
    "Mother's qualification",
    "Father's qualification",
    "Mother's occupation",
    "Father's occupation"
]
X_train_cat = X_train2[cat_cols]
X_train_num = X_train2.drop(columns=cat_cols)

encoder = OneHotEncoder(drop='first', sparse_output=False,␣
↪handle_unknown='ignore')

X_train_cat_enc = encoder.fit_transform(X_train_cat)

X_test_cat = X_test2[cat_cols]
X_test_num = X_test2.drop(columns=cat_cols)
X_test_cat_enc = encoder.transform(X_test_cat)

X_train_final = np.hstack([X_train_num.values, X_train_cat_enc])
X_test_final = np.hstack([X_test_num.values, X_test_cat_enc])

num_cols = X_train_num.columns.tolist()
cat_enc_cols = encoder.get_feature_names_out(cat_cols).tolist()
all_features = num_cols + cat_enc_cols
```

```
    X_train_enc = pd.DataFrame(X_train_final, columns=all_features,␣
↪index=X_train2.index)
    X_test_enc = pd.DataFrame(X_test_final, columns=all_features, index=X_test2.
↪index)
```

```
/home/andreaspanayi8/Documents/machine_learning_exercises
/exercise2/venv/lib64/python3.11/site-
packages/sklearn/preprocessing/_encoders.py:246: UserWarning: Found unknown
categories in columns [5, 6, 7, 8, 9] during transform. These unknown␣
↪categories
will be encoded as all zeros
  warnings.warn(
```

Προβλέψεις

Θα χρησιμοποιήσουμε μοντέλα από τις τρείς ακόλουδες οικογένειες μοντέλων:

1. *Logistic Regression Models* with *Shrinkage Methods*
   i) Logistic Regression
   ii) Logistic Regression + Ridge
   iii) Logistic Regression + Lasso

[11]:
```
#6 Logistic Regression

# 1='Graduated', 0='Dropout'
y_bin = (y_train['Target'] == 'Graduate').astype(int)

# Model 1: raw data
glm = sm.GLM(y_bin, X_train, family=sm.families.Binomial())
LR = glm.fit()
```

Λόγω του μικρού δείγματος (περίπου 3000 εγγραφές) και της μεγάλης διάστασης των features (30 περίπου features), ενδεχεται να πρέπει να χρησιμοποιήσουμε Ridge, Lasso, Elastic Net (ή PCA)

[12]:
```
#7 Logistic Regression with Ridge

# Ridge
C_value = 1.0

# Model 1: raw data with Ridge Regularization
LR_ridge_sklearn = LogisticRegression(
    penalty='l2',
    C=C_value,
    solver='liblinear',
    fit_intercept=False, # Set to False as X_train already includes␣
↪intercept
    random_state=42
)
LR_ridge_sklearn.fit(X_train, y_bin)
```

[12]:
```
LogisticRegression(fit_intercept=False, random_state=42, solver='liblinear')
```

[13]:
```
#8 Logistic Regression with Lasso

# Lasso
C_value = 1.0

# Model 1: raw data with Lasso Regularization
LR_lasso_sklearn = LogisticRegression(
    penalty='l1',
    C=C_value,
    solver='liblinear',
    fit_intercept=False,
    random_state=42
)
LR_lasso_sklearn.fit(X_train, y_bin)
```

[13]:
```
LogisticRegression(fit_intercept=False, penalty='l1', random_state=42,
                   solver='liblinear')
```

Ακρίβεια και απόδωση μοντέλων λογιστικής παλινδρόμησης με μεθόδους συρρίκνωσης

[14]:
```
# Function to evaluate the models
def print_metrics(y_true, y_pred, model_name):
    print(f"Performance for {model_name}:")
    print(f"  Accuracy : {accuracy_score(y_true, y_pred):.4f}")
    print(f"  Precision: {precision_score(y_true, y_pred):.4f}")
    print(f"  Recall   : {recall_score(y_true, y_pred):.4f}")
    print(f"  F1-score : {f1_score(y_true, y_pred):.4f}")
    print()
```

[15]:
```
#9 Models Comparison

# statsmodels predicts probabilities, so get class predictions by␣
↪thresholding at 0.5
pred_probs_lr = LR.predict(exog=X_test)
preds_lr = (pred_probs_lr >= 0.5).astype(int)

# Predict with sklearn Ridge logistic regression
preds_ridge = LR_ridge_sklearn.predict(X_test)

# Predict with sklearn Lasso logistic regression
preds_lasso = LR_lasso_sklearn.predict(X_test)

# Encode 1='Graduate', 0='Dropout'
y_test_bin = (y_test['Target'] == 'Graduate').astype(int)
```

[16]:
```
# Now print results for each model
print_metrics(y_test_bin, preds_lr, "Statsmodels Logistic Regression")
print_metrics(y_test_bin, preds_ridge, "Sklearn Ridge Logistic Regression")
print_metrics(y_test_bin, preds_lasso, "Sklearn Lasso Logistic Regression")

#Confusion Tables
labels_lr    = np.where(preds_lr    == 1, 'Graduate', 'Dropout')
```

```
labels_ridge= np.where(preds_ridge== 1, 'Graduate', 'Dropout')
labels_lasso= np.where(preds_lasso== 1, 'Graduate', 'Dropout')

true_labels = y_test['Target']  # original 'Graduate'/'Dropout' strings

# 1. Statsmodels LR
print("Statsmodels Logistic Regression: ")
display(confusion_table(labels_lr, true_labels))

# 2. Sklearn Ridge
print("\n Sklearn Ridge Logistic Regression: ")
display(confusion_table(labels_ridge, true_labels))

# 3. Sklearn Lasso
print("\n Sklearn Lasso Logistic Regression: ")
display(confusion_table(labels_lasso, true_labels))
```

```
Performance for "Statsmodels" Logistic Regression:
  Accuracy : 0.9118
  Precision: 0.9004
  Recall   : 0.9615
  F1-score : 0.9300

Performance for "Sklearn" Ridge Logistic Regression:
  Accuracy : 0.9146
  Precision: 0.8992
  Recall   : 0.9683
  F1-score : 0.9325

Performance for "Sklearn Lasso" Logistic Regression:
  Accuracy : 0.9174
  Precision: 0.9030
  Recall   : 0.9683
  F1-score : 0.9345

"Statsmodels" Logistic Regression:

Truth      Dropout  Graduate
Predicted
Dropout        237        17
Graduate        47       425


 "Sklearn" Ridge Logistic Regression:

Truth      Dropout  Graduate
Predicted
Dropout        236        14
Graduate        48       428


 "Sklearn" Lasso Logistic Regression:

Truth      Dropout  Graduate
```

```
Predicted
Dropout        238        14
Graduate        46       428
```

2. *Discriminant Analysis*
   - i) Linear Discriminant Analysis (LDA)
   - ii) Quadratic Discriminant Analysis (QDA)

Αρχικά ακολουθεί η προετοιμασία δεδομένων, έπειτα η εκμάθηση των μοντέλων και τέλος ο έλεγχος της ακρίβειας/αποδοτικότητας.

[17]:
```
#11 Data Preparation for Discriminant Analysis

#Remove intercept
X_train_no_int  = X_train.drop(columns=['intercept'])
X_test_no_int   = X_test.drop(columns=['intercept'])

#We can work with labels but as array not data frame
y_train_lbl = y_train['Target']
y_test_lbl  = y_test ['Target']
```

[18]:
```
#12 Linear Discriminant Analysis

#Fit LDA and predict
lda = LDA( store_covariance=True)
lda.fit(X_train, y_train_lbl)

lda_pred = lda.predict(X_test)

#Performance of LDA
display(confusion_table(lda_pred , y_test_lbl))

lda_pred_bin= lda_pred=='Graduate'
y_test_b= y_test_lbl=='Graduate'

print_metrics(y_test_b, lda_pred_bin, "Linear Discriminant Analysis")
```

```
Truth      Dropout  Graduate
Predicted
Dropout        231        10
Graduate        53       432
```

```
Performance for Linear Discriminant Analysis:
  Accuracy : 0.9132
  Precision: 0.8907
  Recall   : 0.9774
  F1-score : 0.9320
```

[19]:
```
# 13 Quadratic Discriminant Analysis

# Fit QDA and predict
qda = QDA(store_covariance=True, reg_param=0.01)
```

```
qda.fit(X_train, y_train_lbl)

qda_pred = qda.predict(X_test)

# Confusion Table
display(confusion_table(qda_pred, y_test_lbl))

# Convert to binary (Graduate = True, Dropout = False)
qda_pred_bin = qda_pred == 'Graduate'
y_test_b = y_test_lbl == 'Graduate'

# Print metrics
print_metrics(y_test_b, qda_pred_bin, "Quadratic Discriminant Analysis")
```

```
Truth      Dropout  Graduate
Predicted
Dropout        230        29
Graduate        54       413
```

```
Performance for Quadratic Discriminant Analysis:
  Accuracy : 0.8857
  Precision: 0.8844
  Recall   : 0.9344
  F1-score : 0.9087
```

3. *Decision Trees*

   i) Random Forest (RF)

   ii) Boosting (GB)

Ομοίως ακολουδεί η προετοιμασία των δεδομένων, η εκμαθηση των μονέλων και ο έλεγχος της ακρίβειας/αποδοτικότητας

[20]:
```
#14 Data Preparation for Tree Models

# Not include intercept
X_Train= X_train_no_int.copy()
X_Test= X_test_no_int.copy()
```

[21]:
```
#15 Random Forest Classifier

# Fit Decision Tree Classifier
rfc=RFC(n_estimators=500,
        max_features=int(math.sqrt(X_Train.shape[1])),
        max_depth=5,
        random_state=0)
rfc.fit(X_Train, y_train_lbl)

# Predict with Random Forest Classifier
rfc_pred = rfc.predict(X_Test)

# Confusion Table
```

```
    display(confusion_table(rfc_pred, y_test_lbl))

    # Convert to binary (Graduate = True, Dropout = False)
    rfc_pred_bin = rfc_pred == 'Graduate'
    y_test_b = y_test_lbl == 'Graduate'

    # Print metrics
    print_metrics(y_test_b, rfc_pred_bin, "Random Forest Classifier")
```

```
Truth      Dropout  Graduate
Predicted
Dropout        228        11
Graduate        56       431

Performance for Random Forest Classifier:
  Accuracy : 0.9077
  Precision: 0.8850
  Recall   : 0.9751
  F1-score : 0.9279
```

[22]:
```
    #16 Random Forest Classifier with enc features

    rfc=RFC(n_estimators=500,
            max_features=int(math.sqrt(X_train_enc.shape[1])),
            max_depth=5,
            random_state=0)
    rfc.fit(X_train_enc, y_train_lbl)

    rfc_Pred = rfc.predict(X_test_enc)

    # Confusion Table
    display(confusion_table(rfc_Pred, y_test_lbl))

    # Convert to binary (Graduate = True, Dropout = False)
    rfc_Pred_bin = rfc_Pred == 'Graduate'

    # Print metrics
    print_metrics(y_test_b, rfc_Pred_bin, "Random Forest Classifier with Encoded␣
↪Features")
```

```
Truth      Dropout  Graduate
Predicted
Dropout        221        21
Graduate        63       421

Performance for Random Forest Classifier with Encoded Features:
  Accuracy : 0.8843
  Precision: 0.8698
  Recall   : 0.9525
  F1-score : 0.9093
```

Η επιλογή των qualitative features για να κωδικοποιηθούν δεν έγινε αποδωτικά, αφού τα αποτελέσματα

φαίνονται να είναι χειρότερα !

[23]:
```
#17 Boosting Classifier

# Fit Gradient Boosting Classifier
gbc = GBC(n_estimators=5000,
          max_depth=5,
          learning_rate=0.01,
          random_state=0)
gbc.fit(X_Train, y_train_lbl)

# Predict with Gradient Boosting Classifier
gbc_pred = gbc.predict(X_Test)

# Confusion Table
display(confusion_table(gbc_pred, y_test_lbl))
# Convert to binary (Graduate = True, Dropout = False)
gbc_pred_bin = gbc_pred == 'Graduate'

# Print metrics
print_metrics(y_test_b, gbc_pred_bin, "Gradient Boosting Classifier")
```

```
Truth      Dropout  Graduate
Predicted
Dropout        229        12
Graduate        55       430
```

```
Performance for Gradient Boosting Classifier:
  Accuracy : 0.9077
  Precision: 0.8866
  Recall   : 0.9729
  F1-score : 0.9277
```

Παρατηρούμε οτι το Boosting επιφέρει ακόμη καλύτερα αποτελέσματα απο το Random Forest

Cross - Validation

Σε αυτό το σημείο θα πρέπει να εκτελέσουμε Cross - Validation ώστε να βρούμε τις κατάλληλες τιμές των *υπερ-παραμέτρων* για το συγκεκριμμένο data set

Αφού το μοντέλο με την καλύτερη ακρίβεια ήταν το Logistic Regression, θα το συνδιάσουμε μαζί με PCA() και Cross - Validation για να βρούμε την βέλτιστη (νέα) διάσταση

[ ]:
```
#18 Logistic Regression with PCA

#libraries
from sklearn.preprocessing   import StandardScaler
from sklearn.decomposition    import PCA
from sklearn.linear_model     import LogisticRegression
from sklearn.model_selection  import GridSearchCV, train_test_split
from sklearn.pipeline         import Pipeline
from sklearn.metrics          import classification_report

pipe = Pipeline([
```

```
    ('scaler', StandardScaler()),
    ('pca',    PCA()),
    ('clf',    LogisticRegression(max_iter=1000)),
])

param_grid = {
    'pca__n_components': [5, 10, 15, 20, 25, 30, 35],
    'clf__C': [0.01, 0.1, 1, 10],
}

grid = GridSearchCV(pipe,
                    param_grid=param_grid,
                    cv=5,
                    scoring='accuracy',
                    n_jobs=-1, #for CPU
                    verbose=1) #for data disp

grid.fit(X_train_no_int, y_bin)

print("Best # components:", grid.best_params_['pca__n_components'])
print("CV accuracy: ", grid.best_score_)

best_pipe = grid.best_estimator_
y_predi=best_pipe.predict(X_test_no_int)

# Confusion Table
l_y_predi = np.where(y_predi == 1, 'Graduate', 'Dropout')

display(confusion_table(l_y_predi, y_test))
```

```
Fitting 5 folds for each of 28 candidates, totalling 140 fits

Best # components: 35
CV accuracy:  0.9094367618256276

Truth      Dropout  Graduate
Predicted
Dropout        238        16
Graduate        46       426
```

Αποτελέσματα Προβλέψεων

Την μεγαλύτερη ακρίβεια φαίνεται να την πετυγχαίνει το μοντέλο *Logistic Regressio + Lasso*

Συνεπώς θα εκτελέσουμε CV για εύρεση των καλύτερων παραμέντρων όσο αφορά την μέθοδο *Logistic Regressio + Lasso*

[ ]:
```
#19 Logistic Regression with Elastic Net

log_reg_elastic = LogisticRegression(
    penalty='elasticnet',
    solver='saga',       # required for elastic net
    l1_ratio=0.5,        # 50% L1, 50% L2
    C=1.0,               # inverse of regularization strength (tune with CV)
```

```
    max_iter=10000        # increase if convergence warnings appear
)

log_reg_elastic.fit(X_train_no_int, y_bin)

y_pred_elastic = log_reg_elastic.predict(X_test_no_int)

# Confusion Table
l_y_pred_elastic = np.where(y_pred_elastic == 1, 'Graduate', 'Dropout')
display(confusion_table(l_y_pred_elastic, y_test))

# Print metrics
print_metrics(y_test_bin, y_pred_elastic, "Logistic Regression with Elastic␣
↪Net")

#CV to find optimal parameters
# param_grid = {'C': [0.01, 0.1, 1, 10, 100]}

# grid = GridSearchCV(LogisticRegression(
#     penalty='elasticnet',
#     solver='saga',
#     l1_ratio=0.5,
#     max_iter=10000),
#     param_grid,
#     cv=5
# )

# grid.fit(X_train, y_train)

# print("Best C:", grid.best_params_)
# print("Best Score:", grid.best_score_)
```

Truth	Dropout	Graduate
Predicted		
Dropout	183	14
Graduate	101	428

```
Performance for Logistic Regression with Elastic Net:
  Accuracy : 0.8416
  Precision: 0.8091
  Recall   : 0.9683
  F1-score : 0.8816
```

[ ]:
```
#19 CV for Logistic Regression with Lasso

#Following the book's lab structure:
outer_cv = ShuffleSplit(n_splits=1, test_size=0.2, random_state=1)
inner_cv = KFold(n_splits=5, shuffle=True, random_state=2)

lambdas = np.logspace(-3, 1, 20)
```

```
enet_cv = ElasticNetCV(
    alphas=lambdas,
    l1_ratio=[0.1, 0.5, 0.9],
    cv=inner_cv,
    max_iter=5000
)

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('enet',   enet_cv),
])

results = cross_validate(
    pipe,
    X_train_no_int, y_bin,
    cv=outer_cv,
    scoring='neg_mean_squared_error',
    return_estimator=True
)

# Best alpha and l1_ratio from the inner loop:
best = results['estimator'][0]

y_pred_best=best.predict(X_test_no_int)
y_pred_class = (y_pred_best >= 0.5).astype(int)

l_y_pred_best = np.where(y_pred_class == 1, 'Graduate', 'Dropout')

#confusion matrix
display(confusion_table(l_y_pred_best, y_test))

# Print metrics
print_metrics(y_test_bin, y_pred_class, "Logistic Regression with Lasso␣
↪(CV)")
```

```
Truth      Dropout  Graduate
Predicted
Dropout        227         9
Graduate        57       433
```

```
Performance for Logistic Regression with Lasso (CV):
  Accuracy : 0.9091
  Precision: 0.8837
  Recall   : 0.9796
  F1-score : 0.9292
```

Ανακεφαλαίωση:

Έχουμε χρησιμοποιήσει:

1. Logistic Regressios + Shrinkage Methods
    - Logistic Regression
    - Logistic Regression + Ridge
    - Logistic Regression + Lasso
2. Discriminant Analysis
    - Linear Discriminant Analysis
    - Quadratic Discriminant Analysis
3. Tree Based Methods
    - Random Forest
    - (Gradient) Boosting
4. Cross Validation
    - CV + PCA
    - CV + ElasticNet

Το καλύτερο ποσοστό επιτυχίας σημείωσε η μέθοδος *Logistic Regression + Lasso* (Elastic Net) με 91.74% ακρίβεια.
Καλή απόδοση επίσης είχε η μέθοδος *LDA*, με 91.32% ακρίβεια.

Άσκηση 2

Η άσκηση αυτή λύθηκε στην τάξη οπότε δεν έχει νόημα να την μεταφράσω σε python.

Αναφορές

[1] Gareth James et al. *An Introduction to Statistical Learning: with Applications in R. with Python code examples*. Adapted Python code available from `https://github.com/...` New York: Springer, 2013. ISBN: 978-1-4614-7138-7.

[2] M. V. Martins et al. "Early prediction of student's performance in higher education: a case study". In: *Trends and Applications in Information Systems and Technologies*. Vol. 1. Advances in Intelligent Systems and Computing. Chapter in Advances in Intelligent Systems and Computing series. Springer, 2021. DOI: 10.1007/978-3-030-72657-7_16.